Unidad 5. Clasificación: cuando Machine Learning tiene que tomar una decisión

5.1. Del número a la decisión

En la unidad anterior terminamos nuestro primer recorrido completo por un problema de Machine Learning utilizando regresión lineal. Partimos de un problema real, identificamos las variables de entrada, definimos aquello que queríamos predecir, entrenamos un modelo y finalmente utilizamos ese modelo para realizar una predicción sobre un caso nuevo. La salida del modelo era un número: el precio estimado de una vivienda.

Ese detalle parece pequeño, pero en realidad marca una diferencia fundamental. No todos los problemas que encontramos en el mundo real consisten en predecir un valor numérico. Muchas veces lo que necesitamos no es saber cuánto vale algo, cuánto tiempo tardará o cuánto dinero se obtendrá, sino **tomar una decisión entre diferentes posibilidades**.

Una entidad bancaria puede necesitar determinar si una operación debería considerarse potencialmente fraudulenta. Un sistema de correo electrónico puede intentar decidir si un mensaje pertenece a la categoría de spam. Una empresa puede querer identificar si un cliente tiene características asociadas con una posible cancelación del servicio. En un sistema médico, determinados datos pueden utilizarse para clasificar un caso dentro de una categoría de riesgo. En todos estos escenarios la pregunta ya no es "¿cuánto?", sino "¿a qué categoría pertenece?".

Esta diferencia nos introduce en el segundo gran problema del aprendizaje supervisado: **la clasificación**.

La idea central de esta clase será comprender que clasificación y regresión comparten una estructura general. En ambos casos tenemos datos históricos, variables de entrada y una variable objetivo. En ambos casos utilizamos ejemplos conocidos para entrenar un modelo. En ambos casos esperamos que ese modelo pueda generalizar y realizar predicciones sobre nuevos ejemplos.

- Lo que cambia es la naturaleza de la respuesta.
- En regresión, la salida es un valor numérico.
- En clasificación, la salida pertenece a una categoría.

Esta distinción va a parecer sencilla al principio, pero tendrá consecuencias importantes cuando lleguemos a la evaluación del modelo. Por eso es necesario construirla con calma.

5.2. El problema de clasificar: un caso de fraude

5.2.1. Un problema real: detectar fraude

Imaginemos ahora una entidad financiera que procesa miles de operaciones diariamente. Cada vez que un cliente utiliza una tarjeta se genera un registro con información relacionada con esa transacción. Podemos tener datos como el importe, el momento de la operación, la ubicación, el tipo de comercio o determinados patrones históricos del cliente.

La entidad posee además información sobre operaciones anteriores que fueron investigadas y clasificadas como fraudulentas o legítimas.

Esto es fundamental.

No solamente tenemos datos sobre las operaciones. También conocemos, al menos para una parte de los casos históricos, cuál fue la categoría correcta.

Podríamos tener algo conceptualmente parecido a esto:

Importe	Hora	Distancia habitual	Comercio	Resultado
$8.500	14:20	2 km	supermercado	legítima
$12.000	15:10	4 km	farmacia	legítima
$450.000	03:15	1.200 km	electrónica	fraude
$7.200	18:30	3 km	supermercado	legítima
$380.000	03:42	950 km	electrónica	fraude

La columna final es diferente de la que teníamos en el ejemplo de regresión. No representa una cantidad continua como el precio de una vivienda. Representa una **categoría**.

Podemos codificarla, por ejemplo, como:

0 → legítima

1 → fraude

Ahora nuestro problema podría formularse de la siguiente manera:

> *Dadas las características de una nueva operación, ¿podemos determinar si pertenece a la categoría "legítima" o a la categoría "fraude"?*

Esta es una pregunta de clasificación.

Y el hecho de que tengamos ejemplos históricos con una respuesta conocida hace que continúe siendo un problema de **aprendizaje supervisado**.

5.2.2. Volvamos al mapa que ya conocemos

En la unidad anterior construimos mentalmente este proceso:



Para clasificación, el mapa sigue siendo prácticamente el mismo.

Lo que cambia es el significado de y.

En regresión:

y = precio

En clasificación:

y = fraude / legítima

Esto nos permite entender una idea fundamental:

regresión y clasificación no son dos mundos completamente separados, son dos tipos de problemas dentro del aprendizaje supervisado.

- En ambos casos el modelo aprende a partir de ejemplos.
- En ambos casos necesitamos variables de entrada.
- En ambos casos existe una variable objetivo.
- En ambos casos queremos generalización.

Lo que cambia fundamentalmente es el tipo de salida que buscamos.

5.3. Qué significa clasificar

5.3.1. ¿Qué significa realmente clasificar?

La palabra "clasificar" puede parecer intuitiva porque la utilizamos constantemente en nuestra vida cotidiana.

- Clasificamos correos.
- Clasificamos documentos.
- Clasificamos productos.
- Clasificamos personas según determinadas características.
- Clasificamos imágenes.

Pero en Machine Learning necesitamos convertir esa idea cotidiana en un problema matemático y computacional.

Supongamos que tenemos dos categorías:

$$\mathbf{C0=legıtima}$$

y

$$\mathbf{C1=fraude}$$

El modelo recibe una observación nueva:

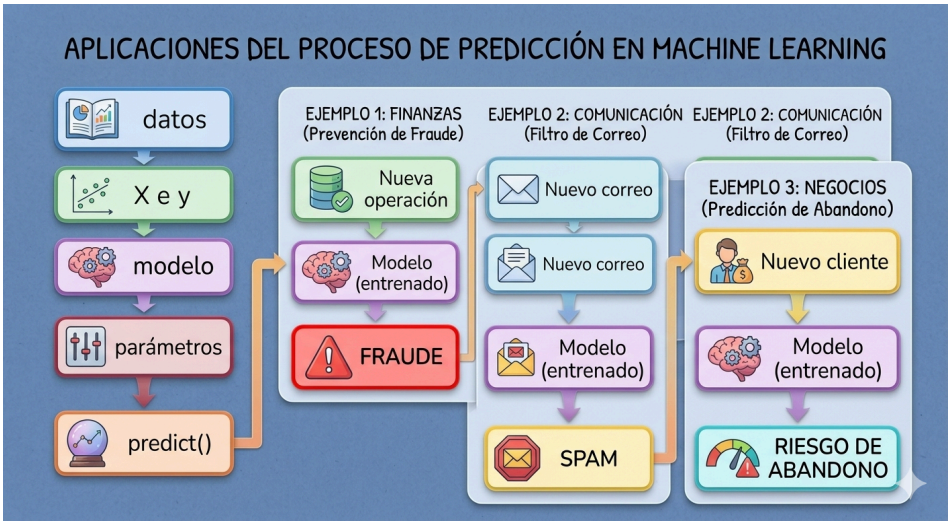
$$\mathbf{Xnuevo}$$

y debe determinar a cuál de las categorías pertenece. Podemos expresarlo conceptualmente como:

$$\mathbf{Xnuevo \rightarrow C}$$

donde C representa la clase asignada.

Por ejemplo:



El modelo está tomando una decisión basada en patrones aprendidos a partir de ejemplos anteriores.

5.3.2. La importancia de las etiquetas

En el aprendizaje supervisado, las categorías conocidas reciben un nombre muy importante: **etiquetas**, o *labels*.

Si tenemos: **operación → fraude**

la palabra "fraude" constituye la etiqueta.

Si tenemos: **correo → spam**

"spam" es la etiqueta.

Si tenemos: **cliente → abandona**

"abandona" es la etiqueta.

Esto vuelve a conectar directamente con lo que estudiamos en las clases anteriores, un conjunto de datos supervisado no solamente contiene características, contiene ejemplos y respuestas conocidas.

Podemos imaginar:

$$\mathbf{X=caracterısticas\ y=etiqueta}$$

Entonces:

$$\mathbf{(X,y)}$$

representa un ejemplo supervisado. En una clasificación binaria, y puede tomar solamente dos valores posibles.

Por ejemplo:

$$\mathbf{y \in \{0,1\}}$$

Pero esos números son solamente una representación computacional.

No debemos confundir:

0 = legítimo

1 = fraude

con una variable numérica continua.

No significa que "fraude" sea una cantidad mayor que "legítimo", los números están funcionando como **códigos de categorías**. Este detalle será importante cuando más adelante trabajemos con diferentes tipos de variables.

5.4. Tipos de clasificación

5.4.1. Clasificación binaria

Cuando tenemos solamente dos categorías posibles hablamos de **clasificación binaria**.

El ejemplo del fraude es binario: **{legıtima,fraude}**

El spam también: **{spam,no spam}**

Una situación de abandono de clientes podría plantearse como: **{abandona,permanece}**

Y un sistema de aprobación podría plantearse como: **{aprobado,rechazado}**

La clasificación binaria es probablemente la forma más sencilla de introducir el concepto porque solamente tenemos que imaginar una decisión entre dos alternativas. Sin embargo, incluso en estos problemas aparentemente sencillos aparecen preguntas importantes. Por ejemplo, imaginemos que un sistema financiero recibe una operación que considera sospechosa.

¿Debe clasificarla directamente como fraude?

¿O debería decir que existe una probabilidad elevada de que sea fraude y dejar la decisión final a una persona?

Esta diferencia entre **predicción** y **decisión operativa** será muy importante.

5.4.2. El modelo no “piensa” en categorías como una persona

Cuando decimos que un modelo clasifica una operación como fraude, podemos caer fácilmente en una interpretación antropomórfica. Parece que el modelo "observó" la operación y "decidió" que era fraude. Pero el modelo no razona de esa manera. Lo que realmente hace es utilizar una función aprendida a partir de datos. Durante el entrenamiento encontró determinados patrones asociados con las diferentes categorías.

Por ejemplo, en un conjunto histórico podría descubrir que determinadas combinaciones de características aparecen con mayor frecuencia entre las operaciones fraudulentas. No significa

que el modelo comprenda el concepto de fraude de la misma manera que una persona. Significa que encontró una relación estadística entre determinadas características y las etiquetas disponibles en los datos de entrenamiento. Esta distinción es fundamental para entender las capacidades y limitaciones del Machine Learning.

5.4.3. Un ejemplo más cercano: abandono de clientes

Veamos ahora otro problema que aparece explícitamente en el enfoque de la materia: la **retención de clientes**, también conocida como *churn*.

Supongamos que una empresa de telecomunicaciones tiene miles de clientes.

Para cada cliente posee información histórica como:

- cantidad de meses utilizando el servicio;
- consumo promedio;
- cantidad de reclamos;
- tipo de plan;
- utilización de determinados servicios;
- historial de pagos.

Además, para clientes históricos sabe cuáles abandonaron la empresa. Entonces podemos construir una variable objetivo:

0 → permanece

1 → abandona

Ahora tenemos un problema de clasificación binaria. La empresa podría entrenar un modelo utilizando los clientes históricos. Posteriormente, cuando aparece un cliente actual, el modelo puede generar una predicción:

Cliente 1452

↓

Modelo

↓

Probabilidad elevada de abandono

La empresa podría utilizar esa información para tomar una acción comercial, por ejemplo, ofrecerle un cambio de plan, un descuento o alguna otra estrategia de retención. Acá aparece una conexión muy importante con un enfoque general.

- El objetivo final no es construir un modelo simplemente porque podemos.

- El objetivo es **transformar un problema de negocio en una solución basada en datos**.

5.4.4. Clasificación multiclase

Hasta ahora utilizamos problemas con dos categorías. Pero no todos los problemas de clasificación son binarios. Imaginemos un sistema que recibe una imagen de un animal y debe determinar si se trata de:

- perro
- gato
- caballo
- ave

Ahora tenemos cuatro categorías.

También podríamos tener un sistema que clasifique un reclamo de soporte técnico como:

- facturación
- conectividad
- usuario
- contraseña
- hardware
- software

Ya no tenemos dos posibilidades.

Tenemos múltiples categorías , a este escenario lo denominamos **clasificación multiclase**.

La lógica general sigue siendo la misma:

$$\mathbf{X \rightarrow C}$$

pero ahora:

$$\mathbf{C \in \{C1, C2, C3, \ldots, Cn\}}$$

El modelo debe determinar a cuál de las categorías disponibles corresponde el nuevo ejemplo. Esto vuelve a mostrar que "clasificación" no significa necesariamente "sí o no". Significa asignar una observación a una categoría perteneciente a un conjunto definido.

5.4.5. Clasificación no es lo mismo que agrupamiento

Supongamos que tenemos clientes. En clasificación supervisada podríamos tener:

- Cliente A → abandona
- Cliente B → permanece
- Cliente C → abandona

Las categorías ya están definidas. El modelo aprende utilizando esas etiquetas. En clustering ocurre algo diferente.

Tenemos los clientes, pero **no conocemos previamente las categorías**. El algoritmo intenta encontrar grupos en los datos.

Por eso:

Clasificación

→ categorías conocidas

→ datos etiquetados

→ aprendizaje supervisado

mientras que:

Clustering

→ categorías desconocidas

→ datos sin etiqueta

→ aprendizaje no supervisado

Esta diferencia será fundamental cuando lleguemos a K-Means y DBSCAN. Por ahora alcanza con conservarla como una idea central.

5.5. La frontera de decisión

5.5.1. ¿Qué significa que una clase esté “separada” de otra?

Imaginemos que representamos dos características de nuestros clientes en un gráfico. En el eje horizontal podemos colocar el consumo mensual. En el eje vertical, la cantidad de reclamos.

Cada cliente aparece como un punto. Además, podemos marcar de una manera diferente los clientes que abandonaron y los que permanecieron. Quizás observemos algo parecido a esto:

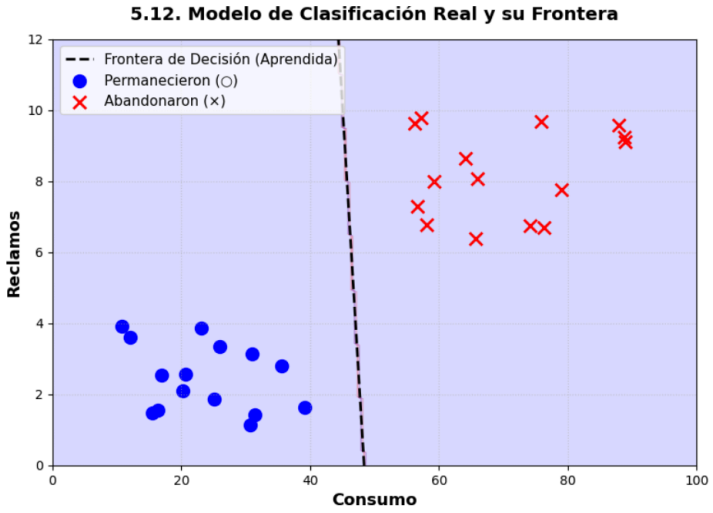


Figura 5.1 — Frontera de decisión aprendida por un modelo de clasificación (consumo vs. reclamos).

Los símbolos ○ representan clientes que permanecieron y las × representan clientes que abandonaron.

Si existe una separación relativamente clara entre ambos grupos, podemos imaginar una frontera que permita dividirlos. Ese concepto de **frontera de decisión** será muy importante para comprender intuitivamente la clasificación. No necesitamos todavía desarrollar la matemática específica de un algoritmo determinado. Lo importante es entender que un modelo de clasificación intenta aprender una regla que permita distinguir las diferentes categorías utilizando las características disponibles.

5.5.2. Análisis de la frontera de decisión

La línea representa, conceptualmente, una **frontera de decisión**. A un lado de la frontera encontramos principalmente observaciones de una categoría. Al otro lado encontramos principalmente observaciones de la otra. Cuando aparece un nuevo cliente, podemos observar en qué región cae y utilizar esa información para asignarle una clase.

Por supuesto, los problemas reales rara vez son tan perfectos.

- Podemos tener puntos mezclados.
- Podemos tener ruido.
- Podemos tener casos ambiguos.
- Podemos tener categorías que se superponen.

Y justamente ahí comienza la verdadera dificultad del aprendizaje automático. Si todos los casos fueran perfectamente separables, construir modelos sería mucho más sencillo.

5.5.3. El mundo real no viene perfectamente separado

Supongamos que tenemos dos grupos de clientes.

En los datos históricos encontramos muchos clientes que abandonaron cuando tenían una gran cantidad de reclamos, pero también encontramos algunos clientes con muchos reclamos que permanecieron.

De la misma manera, podemos encontrar clientes con pocos reclamos que igualmente abandonaron.

¿Qué significa esto?

Significa que las características disponibles **no determinan perfectamente la categoría**.

Y esto es absolutamente normal. Los datos reales contienen incertidumbre. Una persona puede abandonar una empresa por una razón que no está registrada en nuestro dataset. Una operación legítima puede parecerse muchísimo a una operación fraudulenta. Un correo legítimo puede utilizar palabras típicas de un mensaje de spam.

Por eso el Machine Learning no debe interpretarse como una máquina que descubre una verdad absoluta. Es un sistema que aprende patrones a partir de datos y utiliza esos patrones para realizar predicciones.

5.6. Predicción, probabilidad y umbral de decisión

5.6.1. Predicción y p+robabilidad

En muchos problemas de clasificación resulta útil pensar no solamente en una categoría final, sino también en el grado de confianza o probabilidad asociado a la predicción.

Imaginemos una nueva operación bancaria.

El sistema podría producir conceptualmente:

Legítima: 0,15

Fraude: 0,85

Esto significa que el modelo considera que existe una probabilidad estimada de 85% asociada con la clase fraude, según el modelo y los datos utilizados. No significa que el modelo "sepa" que la operación tiene exactamente un 85% de probabilidad de ser fraude en un sentido absoluto.

Es una estimación producida por el modelo. Pero esta información puede ser mucho más útil que simplemente recibir:

FRAUDE

porque permite establecer diferentes estrategias. Por ejemplo, una organización podría decidir:

probabilidad < 0,30 → procesar normalmente

probabilidad 0,30 – 0,70 → solicitar verificación

probabilidad > 0,70 → bloquear o revisar

Los valores concretos dependerán completamente del problema y de las consecuencias de equivocarse.

Esta idea será fundamental cuando estudiemos evaluación de modelos.

5.6.2. El umbral de decisión

Supongamos que nuestro modelo produce:

P(fraude)=0.72, podemos decidir que todo valor superior a 0.50 se clasifique como fraude.

Entonces:

$$0.72 > 0.50$$

y el sistema asigna:

fraude

Pero imaginemos que cambiamos el umbral.

Ahora exigimos:

P(fraude)>0.80, la misma operación, con 0.72, ya no sería clasificada automáticamente como fraude.

Esto demuestra algo muy importante:

> ***La salida de un modelo y la decisión operativa que tomamos a partir de esa salida no necesariamente son exactamente lo mismo.***

Podemos tener una puntuación, una probabilidad estimada o algún valor interno y posteriormente utilizar un criterio para convertirlo en una categoría. Esto será especialmente importante cuando lleguemos a Precision, Recall y F1-Score.

5.6.3. ¿Qué error nos preocupa más?

Volvamos al ejemplo del fraude.

Supongamos que nuestro modelo puede cometer dos errores.

Puede decir:

> *"Esta operación es legítima"*

cuando en realidad era fraudulenta.

O puede decir:

"Esta operación es fraudulenta"

cuando en realidad era legítima.

Ambos son errores, pero probablemente no tengan el mismo costo. Si dejamos pasar una operación fraudulenta, la entidad financiera puede perder una cantidad importante de dinero. Si bloqueamos una operación legítima, podemos generar una mala experiencia para un cliente.

Entonces aparece una pregunta que será central en la próxima clase:

¿Todos los errores tienen el mismo costo?

La respuesta generalmente será no.

Y justamente por eso no podemos evaluar un clasificador utilizando una única medida simplificada. Necesitamos entender **qué tipo de errores está cometiendo**.

5.7. Evaluando un modelo de clasificación: la matriz de confusión

5.7.1. El diagnóstico como problema de clasificación

El ejemplo médico permite entender todavía mejor esta situación. Supongamos que tenemos un sistema que intenta clasificar pacientes en dos categorías:

riesgo

sin riesgo

Imaginemos ahora que el modelo indica:

sin riesgo

pero el paciente realmente pertenece al grupo:

riesgo

El error puede ser especialmente grave. Por otro lado, si el sistema indica:

riesgo

para una persona que finalmente no presenta ese problema, también existe un costo, aunque posiblemente sea de otra naturaleza. Esto nos permite introducir una idea fundamental:

la evaluación de un modelo debe estar relacionada con el contexto de utilización.

No alcanza con decir:

"El modelo tiene 95% de aciertos."

Necesitamos saber qué sucede con el 5% restante.

- ¿Son casos poco importantes?
- ¿Son justamente los casos críticos?
- ¿Son falsos positivos?
- ¿Son falsos negativos?

Esta discusión nos va a llevar directamente a la matriz de confusión.

5.7.2. La matriz de confusión

Para analizar un clasificador necesitamos separar correctamente las diferentes posibilidades. En una clasificación binaria tenemos dos clases reales y dos clases predichas. Eso produce cuatro situaciones fundamentales.

Supongamos nuevamente que la clase positiva es:

fraude

y la negativa:

legítima

Tenemos entonces:

- **Verdadero positivo:** el modelo predijo fraude y realmente era fraude.
- **Verdadero negativo:** el modelo predijo legítima y realmente era legítima.
- **Falso positivo:** el modelo predijo fraude pero la operación era legítima.
- **Falso negativo:** el modelo predijo legítima pero la operación era fraudulenta.

Podemos representarlo:

	Real: fraude	Real: legítima
Predice fraude	Verdadero positivo	Falso positivo
Predice legítima	Falso negativo	Verdadero negativo

Apartir de estas cuatro cantidades podremos construir diferentes métricas. Y cada una responderá una pregunta diferente.

5.7.3. Una pregunta que cambia completamente la evaluación

Imaginemos ahora una base con:

10.000 operaciones.

De ellas solamente:

100 son fraudulentas.

Las otras:

9.900 son legítimas.

Supongamos que construimos un modelo absolutamente inútil que siempre responde:

> *"Legítima".*

No detecta ni un solo fraude.

Sin embargo, acertará: 9900

operaciones sobre: 10000

Es decir, tendría un porcentaje de aciertos del: 99%

¿Podríamos decir que es un excelente detector de fraude?

Evidentemente no. Y este ejemplo es extremadamente importante.

Porque demuestra que una métrica aparentemente intuitiva puede resultar engañosa cuando las clases están desbalanceadas. Este será uno de los motivos por los cuales necesitamos estudiar otras métricas además del porcentaje global de aciertos.

5.8. Métricas de evaluación: precisión, recall y F1

5.8.1. El problema del desbalance de clases

El ejemplo anterior introduce un concepto que debemos conocer: **desbalance de clases**.

Tenemos:

99% legítimas

1% fraude

La clase legítima es muchísimo más frecuente que la clase fraude. Esto ocurre en numerosos problemas reales. En detección de fraude, los fraudes suelen representar una pequeña proporción del total. En determinadas aplicaciones médicas, la condición que queremos detectar puede ser relativamente poco frecuente. En detección de fallas industriales, la mayoría de los equipos funcionan correctamente y solamente una pequeña parte presenta fallas.

Esto genera una dificultad:

> *un modelo puede obtener un porcentaje elevado de aciertos simplemente porque la clase mayoritaria domina el conjunto de datos.*

Por eso debemos mirar más allá del número global.

No debemos preguntar solamente:

> *"¿Cuántas predicciones acertó?"*

También debemos preguntar:

> *"¿Qué tipo de casos está acertando y cuáles está fallando?"*

Esta pregunta nos prepara directamente para Precision y Recall.

5.8.2. Precision: cuando importa la calidad de los positivos

Supongamos que nuestro modelo marca 100 operaciones como fraudulentas. Pero cuando las investigamos descubrimos que solamente 40 eran realmente fraude, las otras 60 eran operaciones legítimas. Podemos decir que el modelo fue demasiado agresivo. Estaba señalando como fraude demasiadas operaciones que en realidad eran legítimas. La **Precision** responde conceptualmente a esta pregunta:

> ***De todos los casos que el modelo predijo como positivos, ¿cuántos eran realmente positivos?***

La fórmula es:

$$Precision = \frac{TP}{TP + FP}$$

donde:

- TP representa verdaderos positivos;
- FP representa falsos positivos.

Volvamos al ejemplo.

Tenemos:

TP=40

y:

FP=60

Entonces:

$$Precision = \frac{40}{40 + 60}$$

$$Precision = 0.40$$

Es decir:

Precision=40%

Esto significa que solamente cuatro de cada diez operaciones que el modelo marcó como fraude realmente lo eran.

5.8.3. Recall: cuando importa no dejar escapar positivos

Ahora imaginemos otra situación. Sabemos que existen 100 operaciones fraudulentas, nuestro modelo consigue detectar 90, pero deja pasar 10. En este caso queremos medir otra cosa.

La pregunta es:

> ***De todos los positivos que realmente existían, ¿cuántos consiguió encontrar el modelo?***

Eso es **Recall**.

Su fórmula es:

$$Recall = \frac{TP}{TP + FN}$$

Si:

TP=90

y:

FN=10

entonces:

$$Recall = \frac{90}{90 + 10} = 0.90$$

Tenemos:

Recall=90%

El modelo consiguió encontrar nueve de cada diez fraudes reales. Esta diferencia entre Precision y Recall es fundamental.

- Precision mira desde el punto de vista de las predicciones positivas realizadas.
- Recall mira desde el punto de vista de los positivos que realmente existían.

5.8.4. Precision y Recall cuentan historias diferentes

Imaginemos dos sistemas de detección de fraude.

El primero es extremadamente conservador. Solamente marca una operación como fraude cuando está muy seguro.

Podría detectar:

20 fraudes

y acertar:

19

Tendría una Precision muy alta, pero quizás existían 100 fraudes en total, entonces solamente encontró 19 de 100, su Recall sería muy bajo.

El segundo sistema podría marcar muchísimas operaciones como sospechosas. Tal vez detecte:

90 de los 100 fraudes

pero también marque como fraudulentas muchas operaciones legítimas, entonces podría tener un Recall alto pero una Precision mucho menor, ninguno de los dos modelos es automáticamente "mejor", depende de lo que necesitemos y acá aparece una de las ideas más importantes:

> ***La evaluación de un modelo no puede separarse del problema que queremos resolver.***

5.8.5. El caso del diagnóstico

Volvamos al ejemplo médico.

Si estamos intentando detectar una enfermedad grave, quizás sea particularmente importante minimizar los casos en los que una persona enferma sea clasificada como sana. Eso significa que nos preocupa mucho el **falso negativo**.

En ese contexto puede ser especialmente importante obtener un Recall elevado, pero eso no significa que Precision sea irrelevante, si el sistema marca prácticamente a todo el mundo como enfermo, tendremos un problema diferente. Muchísimas personas sanas serían enviadas a estudios innecesarios, generando costos y posiblemente ansiedad. Por eso en los problemas reales normalmente buscamos un equilibrio y ese equilibrio nos lleva a la siguiente métrica.

5.8.6. F1-Score: buscando un equilibrio

Cuando necesitamos combinar Precision y Recall podemos utilizar el **F1-Score**.

La fórmula es:

$$F1 = 2\frac{Precision \cdot Recall}{Precision + Recall}$$

No necesitamos convertir esta fórmula en el centro matemático de la clase. Lo importante es comprender qué intenta hacer.

- Si tenemos una Precision muy alta pero un Recall muy bajo, el F1-Score reflejará ese desequilibrio.
- Si tenemos un Recall muy alto pero una Precision muy baja, también.

Para obtener un F1 alto necesitamos que ambas métricas sean razonablemente buenas. Por eso resulta útil cuando queremos tener una medida que considere simultáneamente ambos aspectos del comportamiento del clasificador.

Supongamos que tenemos dos modelos para detectar fraude.

El modelo A tiene:

Precision=95% Recall=40%

El modelo B tiene:

Precision=75% Recall=90%

¿Cuál es mejor?

Primero hay que preguntar:

> *¿Qué problema estamos tratando de resolver?*

- Si la prioridad absoluta fuera no bloquear operaciones legítimas, quizás el modelo A resulte atractivo.
- Si la prioridad fuera detectar prácticamente todos los fraudes, probablemente el modelo B sea más interesante.

Entonces aparece una conclusión que vale oro:

> ***No existe una métrica universal que permita declarar ganador a un modelo independientemente del problema.***

La métrica debe interpretarse en función del contexto.

5.9. De las métricas a la decisión de negocio

5.9.1. Clasificación como problema de negocio

Hasta acá podríamos pensar que estamos haciendo solamente matemática, pero justamente el enfoque de esta materia busca evitar eso. Supongamos que una empresa tiene un sistema que clasifica clientes según riesgo de abandono.

El modelo obtiene:

Recall=90%

Puede parecer un excelente resultado, pero la pregunta importante es:

> *¿Qué significa ese 90% en el funcionamiento real de la empresa?*

Si significa que conseguimos identificar nueve de cada diez clientes que efectivamente iban a abandonar, entonces puede tener un valor comercial enorme.

La empresa puede actuar sobre esos clientes.

Pero si para conseguir ese Recall tuvimos que contactar a millones de clientes que nunca pensaban abandonar, quizá el costo de la estrategia sea demasiado alto.

Por eso las métricas técnicas deben poder traducirse en consecuencias operativas.

Este principio aparece explícitamente en el enfoque de la materia: evaluar la performance técnica de los modelos y relacionar esos resultados con el valor de negocio.

5.9.2. Del modelo a una aplicación

Pensemos ahora como programadores.

Supongamos que tenemos una aplicación web de una entidad financiera.

- El usuario realiza una operación.
- El backend recibe los datos.
- Nuestro modelo de Machine Learning está entrenado.

La aplicación puede construir un vector de características:

1. importe
2. hora
3. distancia
4. tipo_comercio

...

- Ese vector se envía al modelo.
- El modelo produce una predicción.
- La aplicación puede entonces tomar una decisión:

operación normal

o:

operación sospechosa

Esto es exactamente el tipo de escenario que se busca, no se trata solamente de estudiar algoritmos de manera aislada.

El objetivo es que el programador pueda **integrar modelos de Machine Learning dentro de soluciones de software reales**.

Por eso, cuando trabajemos con Python y Scikit-Learn, no debemos pensar en el modelo como algo separado del sistema.

El modelo es un componente dentro de una arquitectura mayor.

5.9.3. ¿Qué aprendimos realmente en esta primera parte?

Llegados hasta acá ya podemos construir una diferencia muy clara entre regresión y clasificación. En regresión, como vimos en la clase anterior, queremos obtener un valor numérico:

$X \rightarrow y$

donde y puede ser:

1. precio
2. temperatura
3. ventas
4. tiempo
5. consumo

En clasificación tenemos:

$\mathbf{X \rightarrow C}$

donde C representa una categoría:

- fraude

- spam
- abandona
- riesgo
- legítimo

En ambos casos tenemos aprendizaje supervisado porque contamos con ejemplos históricos y conocemos la respuesta esperada durante el entrenamiento.

Pero en clasificación aparece una nueva dimensión: **no alcanza con saber si acertamos o fallamos; necesitamos comprender qué tipo de errores estamos cometiendo**.

Y justamente por eso aparecen:

TP, TN, FP, FN

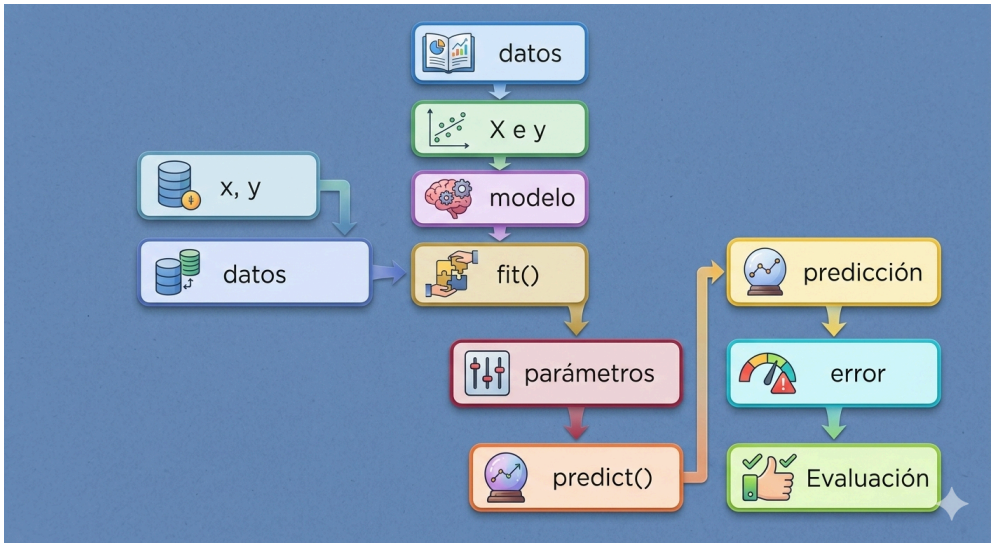
y a partir de ellos:

Precision Recall F1

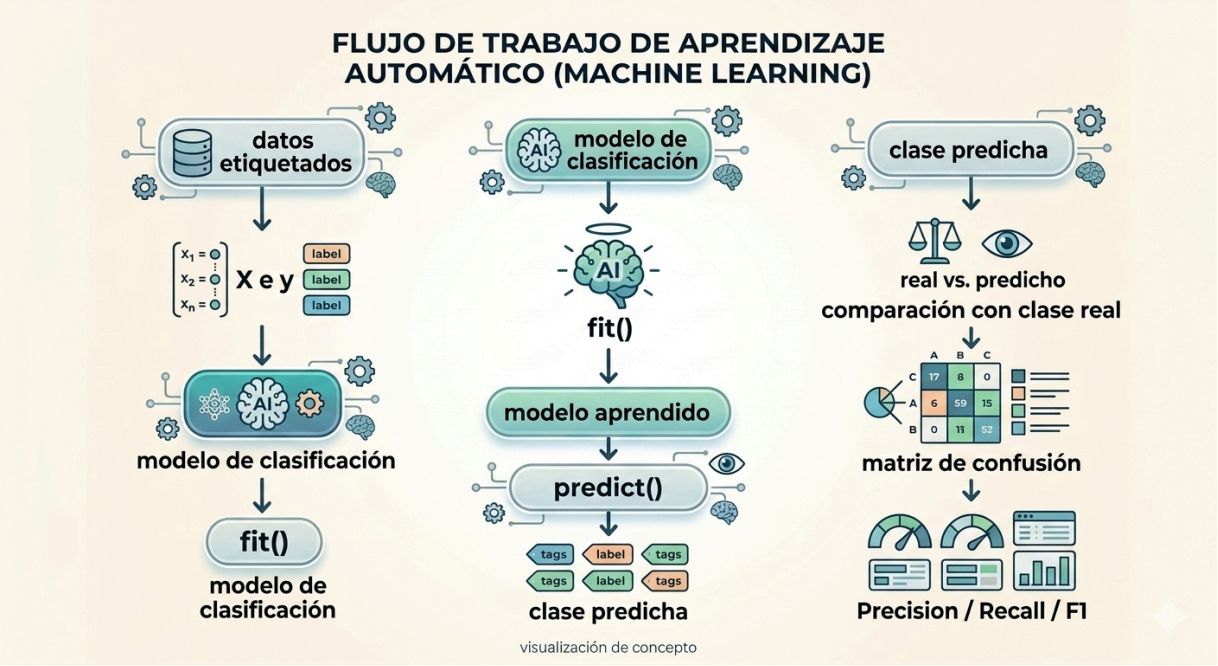
Estas métricas no son simplemente fórmulas que vamos a memorizar. Representan preguntas diferentes sobre el comportamiento del modelo.

5.9.4. Una última conexión

Quiero cerrar esta primera parte recuperando algo que hicimos al final de la regresión lineal. En aquella unidad construimos este recorrido:



Ahora podemos hacer exactamente lo mismo para clasificación:



Esto nos muestra algo que quiero que quede muy claro.

La clasificación no aparece como un tema completamente nuevo.

Es una extensión natural del camino que ya construimos. Lo nuevo es que ahora la respuesta no es un número continuo sino una categoría, y por eso también cambia la forma en que evaluamos el resultado.

5.10. El rol humano en la decisión final

5.10.1. El verdadero desafío: elegir bien qué significa “positivo”

Hay una cuestión que muchas veces se pasa por alto cuando se explica clasificación y que conviene trabajar desde ahora. Cuando hablamos de "positivo" y "negativo", no estamos diciendo que una clase sea necesariamente buena y la otra mala.

Estamos simplemente asignando un nombre técnico a una de las categorías.

Por ejemplo, en fraude podemos definir:

positivo = fraude

negativo = legítima

Pero podríamos tener otro problema donde:

positivo = cliente que permanece

negativo = cliente que abandona

Lo importante es establecer claramente cuál consideramos la clase positiva antes de interpretar Precision, Recall y las demás métricas. Esto también demuestra que las métricas no existen independientemente del problema. La forma en que definimos las clases influye en la interpretación de los resultados.

5.10.2. Cuando el modelo se equivoca, necesitamos mirar el caso

Supongamos que nuestro modelo comete un falso positivo.

La predicción fue:

fraude

pero la operación realmente era:

legítima

Una forma superficial de analizarlo sería simplemente sumar uno a la cantidad de errores.

Pero desde una perspectiva de ingeniería eso no es suficiente.

Queremos preguntarnos:

1. ¿Por qué ocurrió?
2. ¿El importe era inusualmente elevado?
3. ¿La ubicación era diferente?
4. ¿El cliente había realizado operaciones similares anteriormente?
5. ¿Existía algún dato faltante?
6. ¿El conjunto de entrenamiento tenía suficientes ejemplos de este tipo?

Esta forma de pensar es importante porque evaluar Machine Learning no significa solamente calcular una métrica.

También significa **comprender el comportamiento del sistema**.

Y esto conecta directamente con el perfil profesional que busca la materia: el estudiante debe poder interpretar datos, seleccionar modelos y justificar técnicamente una solución.

5.10.3. El modelo no reemplaza automáticamente la decisión humana

Otro punto importante aparece especialmente en aplicaciones sensibles. Supongamos que un modelo determina que una operación tiene una alta probabilidad de fraude. Eso no significa necesariamente que debamos bloquearla automáticamente. Podemos diseñar diferentes arquitecturas.



Esto es importante porque demuestra que Machine Learning puede ser solamente **una pieza dentro de un sistema mayor**.

- El modelo produce información.
- La aplicación utiliza esa información.
- Las reglas de negocio pueden combinarla con otras condiciones.
- Y finalmente se produce una acción.

Con la clasificación damos un paso importante dentro del aprendizaje supervisado. En la regresión lineal aprendimos a utilizar datos históricos para estimar un valor numérico. Ahora mantenemos la misma lógica general, pero cambiamos la naturaleza de la respuesta: queremos determinar a qué categoría pertenece un nuevo ejemplo.

El cambio parece sencillo, pero introduce una problemática mucho más rica. Cuando un modelo predice un precio podemos medir cuánto se alejó del valor real. En clasificación, en cambio, una predicción puede ser correcta o incorrecta, pero los errores no necesariamente tienen todos el mismo significado. Confundir una operación legítima con fraude no es exactamente el mismo problema que dejar pasar una operación fraudulenta. Confundir un paciente sano con uno de riesgo tampoco tiene necesariamente las mismas consecuencias que el error contrario.

Por eso comenzamos a mirar el problema desde otra perspectiva. Ya no alcanza con preguntar cuántas predicciones acertó el modelo. Necesitamos analizar **qué está acertando, qué está fallando y qué tipo de error resulta más importante para nuestro problema**.

Esa necesidad nos lleva naturalmente a la matriz de confusión y a las métricas de Precision, Recall y F1-Score.

> ***Un modelo de Machine Learning no se evalúa solamente por lo bien que predice.***
> ***Se evalúa por lo bien que resuelve el problema para el cual fue construido.***

En la siguiente parte vamos a profundizar justamente en esa idea. Vamos a tomar la matriz de confusión y construirla desde cero con ejemplos numéricos, porque ahí es donde Precision, Recall y F1 dejan de ser tres fórmulas para memorizar y empiezan a tener un significado concreto.

Después vamos a llevarlo nuevamente a Python y veremos cómo un modelo puede parecer excelente si solamente miramos el porcentaje de aciertos y, sin embargo, ser prácticamente inútil para detectar aquello que realmente nos interesa.

5.11. Generalización: aprender más allá de los datos de entrenamiento

5.11.1. Del rendimiento sobre los datos al comportamiento sobre datos nuevos

Hasta este punto evaluamos qué tan bien se comporta un modelo cuando comparamos sus predicciones con los resultados reales. Sin embargo, todavía queda una pregunta fundamental: **¿sobre qué datos estamos realizando esa evaluación?**

Esta pregunta es importante porque un modelo de Machine Learning aprende a partir de los datos que le proporcionamos. Si utilizamos exactamente esos mismos datos para entrenarlo y después afirmamos que funciona bien porque consiguió acertar la mayoría de ellos, estamos realizando una evaluación incompleta.

Imaginemos, por ejemplo, que queremos construir un modelo capaz de reconocer imágenes de perros y gatos. Le proporcionamos miles de fotografías para que aprenda y, una vez terminado el entrenamiento, le presentamos nuevamente esas mismas fotografías. Es posible que consiga un resultado ex+celente. Pero eso no necesariamente significa que haya aprendido a reconocer perros y gatos de manera general.

Podría haber aprendido características demasiado específicas de las imágenes que utilizamos durante el entrenamiento.

Por ejemplo, quizás todas las fotografías de perros utilizadas durante el entrenamiento fueron tomadas en exteriores y muchas fotografías de gatos fueron tomadas dentro de una casa. Un modelo podría encontrar accidentalmente esa relación y utilizar el fondo como una señal para clasificar las imágenes.

Cuando aparezca una fotografía de un perro dentro de una casa, podría equivocarse.

El modelo habría aprendido algo presente en los datos de entrenamiento, pero no necesariamente aquello que realmente nos interesaba.

Por eso aparece un concepto fundamental:

> ***Un modelo debe ser evaluado por su capacidad para generalizar a datos que no utilizó durante su aprendizaje.***

5.11.2. Generalización

En Machine Learning llamamos **generalización** a la capacidad de un modelo para mantener un buen desempeño cuando recibe datos nuevos, diferentes de aquellos utilizados durante el entrenamiento.

Esta idea permite distinguir entre dos comportamientos muy diferentes.

Un modelo puede aprender correctamente los patrones generales presentes en los datos y luego utilizarlos para realizar buenas predicciones sobre nuevos ejemplos.

Pero también puede aprender características demasiado específicas de los datos disponibles, incluyendo ruido, excepciones o particularidades que no se repiten en otros casos.

- En el primer escenario hablamos de una buena capacidad de generalización.
- En el segundo aparece uno de los problemas más conocidos de Machine Learning: **overfitting**.

5.12. Overfitting y underfitting

5.12.1. Overfitting: cuando el modelo aprende demasiado

El **overfitting**, o sobreajuste, ocurre cuando un modelo se adapta excesivamente a los datos de entrenamiento.

Esto significa que consigue representar muy bien los ejemplos que utilizó para aprender, pero tiene dificultades cuando recibe datos nuevos.

Podemos imaginarlo de una manera sencilla. Supongamos que un estudiante recibe una lista de cien ejercicios para preparar un examen. En lugar de comprender cómo resolverlos, memoriza exactamente las respuestas de esos cien ejercicios.

Cuando el profesor vuelve a colocar los mismos ejercicios, el estudiante obtiene una calificación excelente. Pero si cambia los números o plantea un problema ligeramente diferente, el rendimiento cae considerablemente. El estudiante no aprendió realmente el procedimiento general. Aprendió los ejemplos particulares. Con un modelo de Machine Learning puede ocurrir algo similar.

La diferencia es que el modelo no "memoriza" necesariamente de la misma manera que una persona, pero puede ajustarse tanto a los datos de entrenamiento que termina capturando también características que no representan el patrón general.

5.12.2. ¿Cómo reconocemos un overfitting?

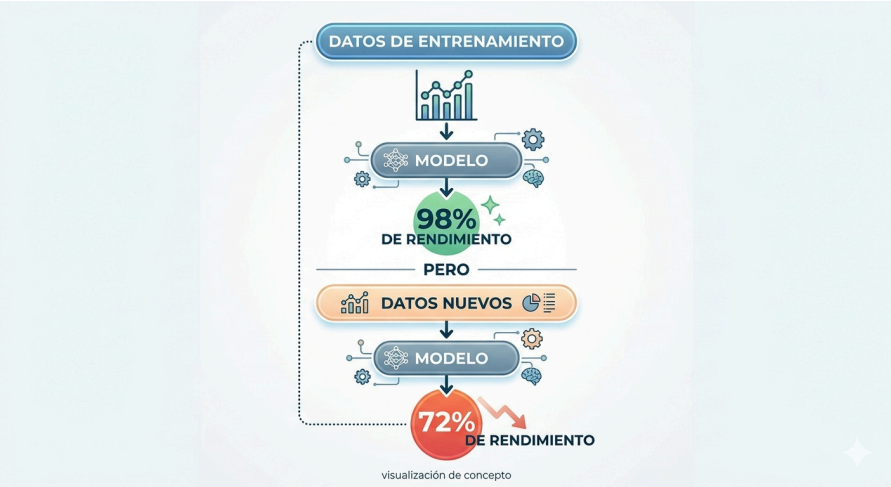
Una de las señales más claras aparece cuando observamos una diferencia importante entre el rendimiento obtenido sobre los datos de entrenamiento y el obtenido sobre datos que el modelo no utilizó para aprender.

Por ejemplo:

Conjunto	Rendimiento
Entrenamiento	98 %
Prueba	72 %

El modelo parece funcionar extraordinariamente bien durante el entrenamiento, pero su rendimiento disminuye considerablemente cuando se enfrenta a nuevos datos.

Esto debería hacernos sospechar de un posible **sobreajuste**, la idea puede representarse de manera muy sencilla:



La diferencia es demasiado grande.

El modelo aprendió muy bien los datos que conocía, pero no logró aprender una representación suficientemente general del problema.

5.12.3. Underfitting: cuando el modelo aprende demasiado poco

El problema contrario es el **underfitting**, o subajuste. En este caso el modelo es demasiado simple, está insuficientemente entrenado o no dispone de una representación adecuada para capturar las relaciones existentes en los datos, podemos tener, por ejemplo:

Conjunto	Rendimiento
Entrenamiento	62 %
Prueba	60 %

Acá no existe una diferencia enorme entre entrenamiento y prueba, sin embargo, ambos resultados son malos. El modelo no está aprendiendo suficientemente bien, por eso podemos pensar en tres situaciones generales:

UNDERFITTING

Situación	Rendimiento
Aprende poco	Mal rendimiento en entrenamiento y mal rendimiento en prueba

BUENA GENERALIZACIÓN

Situación	Rendimiento
Aprende el patrón	Buen rendimiento en entrenamiento y buen rendimiento en prueba

OVERFITTING

Situación	Rendimiento
Aprende demasiado los datos	Excelente entrenamiento, mal rendimiento en prueba

Esta comparación es mucho más importante que memorizar las definiciones aisladas.

5.12.4. Complejidad del modelo

Una de las causas relacionadas con el overfitting es la **complejidad del modelo**. Un modelo demasiado simple puede ser incapaz de representar correctamente el problema., pero un modelo excesivamente complejo puede adaptarse demasiado a los datos disponibles, podemos imaginarlo como intentar dibujar una línea que separe dos grupos de puntos, una separación demasiado simple puede cometer muchos errores.

Pero una frontera excesivamente compleja podría intentar pasar exactamente entre cada punto del conjunto de entrenamiento, generando una solución que funciona perfectamente sobre esos ejemplos pero que se comporta mal ante nuevos datos.

Por eso, en Machine Learning no buscamos necesariamente el modelo más complejo.Buscamos un modelo con una complejidad adecuada para el problema.

5.13. Entrenamiento, validación y prueba

5.13.1. Entrenamiento, validación y prueba

Para evaluar correctamente un modelo necesitamos separar las funciones que cumplen los datos. Una división habitual consiste en trabajar con tres conjuntos:

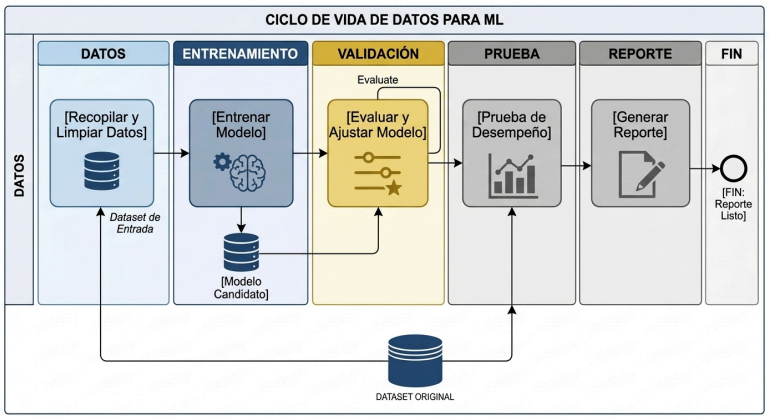
entrenamiento, **validación** y **prueba**.

El conjunto de entrenamiento se utiliza para que el modelo aprenda sus parámetros.

El conjunto de validación permite comparar configuraciones y tomar decisiones sobre el modelo durante el desarrollo.

Finalmente, el conjunto de prueba se reserva para realizar una evaluación final sobre datos que el modelo no utilizó durante el proceso de ajuste.

Conceptualmente:



Esta separación evita que confundamos la capacidad del modelo para aprender los datos disponibles con su capacidad para generalizar.

5.13.2. ¿Por qué no usamos solamente entrenamiento y prueba?

Podríamos preguntarnos por qué necesitamos un conjunto de validación. Supongamos que tenemos tres modelos diferentes y queremos elegir uno.

Probamos los tres sobre el conjunto de prueba y elegimos aquel que obtiene el mejor resultado.

El problema es que, si repetimos este procedimiento muchas veces, comenzamos indirectamente a adaptar nuestras decisiones al conjunto de prueba.

El conjunto que debía representar datos desconocidos deja de ser completamente independiente de nuestro proceso de selección.

El conjunto de validación permite realizar esas decisiones durante el desarrollo y reservar el conjunto de prueba para una evaluación final.

No es necesario que el estudiante memorice una proporción específica de división para comprender este concepto. Lo importante es entender **la función que cumple cada conjunto**.

5.13.3. ¿Cómo podemos mejorar un modelo con overfitting?

Detectar overfitting no significa que tengamos que descartar inmediatamente el modelo.

Existen diferentes estrategias para reducirlo. Una de ellas consiste en **reducir la complejidad del modelo**. Si el modelo tiene demasiada capacidad para adaptarse a los datos, utilizar una estructura más sencilla puede favorecer la generalización.

Otra posibilidad consiste en trabajar sobre las características utilizadas. No todas las variables disponibles necesariamente aportan información útil. Algunas pueden introducir ruido o relaciones accidentales.

También podemos utilizar técnicas de **regularización**, que agregan restricciones al proceso de aprendizaje para evitar que el modelo adopte soluciones excesivamente complejas.

En otros casos podemos necesitar más datos de entrenamiento. Si el modelo dispone de pocos ejemplos, puede resultar más fácil que se adapte demasiado a particularidades de esos datos. La estrategia concreta dependerá del algoritmo utilizado y del problema.

5.13.4. ¿Y cómo solucionamos el underfitting?

Cuando tenemos underfitting, el problema es diferente. El modelo no está capturando suficientemente bien la estructura de los datos.

Podemos necesitar un modelo más expresivo, mejores características o un proceso de entrenamiento más adecuado. También puede ocurrir que el problema sea simplemente que estamos utilizando una representación demasiado pobre de los datos.

Por ejemplo, imaginemos que queremos predecir el precio de una vivienda utilizando únicamente la cantidad de habitaciones. Probablemente esa variable sea importante, pero difícilmente alcance para explicar completamente el precio.

La superficie, ubicación, antigüedad, estado de conservación y otras características podrían aportar información adicional. Por lo tanto, mejorar el modelo no significa siempre cambiar el algoritmo. A veces necesitamos **mejores datos o mejores características**.

5.13.5. El equilibrio entre aprender y generalizar

Podemos pensar entonces que construir un modelo de Machine Learning implica buscar un equilibrio.

- Si el modelo es demasiado sencillo, puede no aprender suficientemente bien.

- Si es demasiado complejo, puede adaptarse demasiado a los datos de entrenamiento.

Entre ambos extremos buscamos una solución que capture los patrones relevantes y descarte, en la medida de lo posible, el ruido y las particularidades que no se generalizan. Esta idea está relacionada con el clásico equilibrio entre **sesgo y varianza**, que permite comprender de manera más profunda el comportamiento de los modelos. El sesgo representa, de manera conceptual, el error asociado a utilizar un modelo demasiado simple para representar el problema. La varianza representa la sensibilidad excesiva del modelo a las particularidades de los datos de entrenamiento.

Podemos relacionarlo con lo que acabamos de estudiar:

Modelo demasiado simple

↓

Mayor sesgo

↓

Underfitting

Modelo demasiado sensible

↓

Mayor varianza

↓

Overfitting

5.14. Cierre de la unidad

5.14.1. ¿Qué aprendimos realmente en esta etapa?

Si miramos el recorrido completo de las últimas unidades, podemos observar que hemos pasado por varias preguntas diferentes.

Primero preguntamos:

¿Qué queremos predecir?

Eso nos permitió diferenciar regresión y clasificación.

Después preguntamos:

¿Cómo aprende el modelo?

Ahí trabajamos con datos de entrenamiento y aprendizaje supervisado.

Luego apareció otra pregunta:

¿Cómo sabemos si el modelo funciona?

Ahí introdujimos las métricas de evaluación.

Y finalmente llegamos a una pregunta todavía más importante:

¿Funciona solamente con los datos que conoce o también con datos nuevos?

Esta última pregunta nos llevó a la generalización, al overfitting y al underfitting.

5.14.2. Un ejemplo sencillo para cerrar

Supongamos que construimos tres modelos para clasificar imágenes.

El primero obtiene:

Entrenamiento: 65 %

Prueba: 63 %

El segundo:

Entrenamiento: 89 %

Prueba: 86 %

El tercero:

Entrenamiento: 99 %

Prueba: 70 %

El primero probablemente presenta **underfitting**: no consigue aprender suficientemente bien.

El tercero presenta una señal clara de **overfitting**: funciona casi perfectamente sobre entrenamiento, pero pierde mucho rendimiento sobre datos nuevos.

El segundo presenta, en principio, un comportamiento más equilibrado.

No podemos afirmar solamente con estos números que sea "el mejor modelo" para cualquier situación, porque todavía debemos considerar las métricas concretas del problema, el costo de los errores y otros factores.

Pero sí podemos afirmar que presenta una **generalización más razonable**.

Con esto completamos el bloque de evaluación de modelos de clasificación.

Comenzamos analizando cómo evaluar las predicciones mediante la matriz de confusión y métricas como Precision, Recall y F1-Score. Estas herramientas nos permitieron comprender que el rendimiento de un modelo no puede reducirse simplemente a contar cuántas predicciones fueron correctas.

Después dimos un paso más y analizamos qué ocurre cuando evaluamos un modelo sobre datos nuevos. Allí apareció el concepto de generalización y comprendimos que un modelo debe

aprender patrones que puedan mantenerse fuera del conjunto utilizado durante su entrenamiento.

A partir de esta idea diferenciamos **underfitting**, cuando el modelo resulta demasiado simple o no consigue aprender suficientemente bien, y **overfitting**, cuando se adapta excesivamente a los datos de entrenamiento y pierde capacidad de generalización.

También introdujimos la separación entre entrenamiento, validación y prueba, entendiendo que cada conjunto cumple una función diferente dentro del desarrollo y evaluación de un modelo.

El objetivo de toda esta etapa no es encontrar un modelo que simplemente tenga un número elevado en una métrica. El objetivo es construir un modelo que **aprenda adecuadamente, generalice sobre datos nuevos y resulte útil para el problema real que queremos resolver**.